

How Traditional software Engineers Scaffold a Feature

1. **Scaffolding as code generation** — running a tool that spits out boilerplate structure so you don't hand-write it: ``rails generate``, ``nest g resource``, ``php artisan make:model``, ``npx create-next-app``, a Flutter package template, Prisma/Drizzle schema-to-migration. This gives you empty-but-wired files.

2. **Scaffolding as skeleton-building** — standing up a thin, end-to-end skeleton of the feature (the **vertical slice**) that runs but does almost nothing, then progressively filling it in. This is the more important skill; the generators just save typing inside it.

Phase 0 — Frame the feature before touching code

You cannot scaffold what you haven't defined.

Write the problem statement: what user pain does this solve, and how do we know it's solved?

Not the solution — the problem.

Write it as a user story with acceptance criteria.

Eg. "As a **shop owner**, I can **see this month's revenue**, so that **I know if I'm profitable**."

Then the criteria: the concrete, testable conditions that mean "done" (e.g. **totals exclude refunded orders**, **empty state shows a prompt**, **loads in under 2s for 10k rows**).

These criteria become your tests later — write them now while your thinking is honest.

Define non-goals explicitly. This is a critical scope-control move. "This does **not** include multi-currency, does **not** include export." Non-goals stop the feature from silently tripling in size.

Identify unknowns and spike them. If any part is "I don't actually know how X works" (a third-party API, an unfamiliar library, a performance question), do a **spike**: a small throwaway experiment, timeboxed, to answer the question. You throw the spike code away and keep the knowledge. Never scaffold on top of an unanswered unknown.

Decide success metrics. if it's a product feature — what you'll look at after release to know it worked.

Phase 1 — Technical design

Translate intent into a plan across the layers of the system.

Model the data- What entities exist, their fields and types, their relationships (one-to-many, many-to-many), constraints (unique, not-null, foreign keys), and indexes you'll need for the queries you plan to run.

>The data model is the spine; get it wrong and every layer above it inherits the mistake.

>Sketch the migration mentally: is this additive (safe) or does it alter/drop existing columns (dangerous, needs a plan)?

Design the API contract / interface. The shape of the boundary between backend and frontend (or between modules): endpoints or method signatures, request payloads, response shapes, status codes, error formats.

>Define this **before** implementing either side so both can be built against a fixed contract. This is where you decide the "shape" the frontend will consume.

Sketch the UI / component tree. This is What screens/components, what state each holds, what it fetches, the loading/empty/error states (not just the happy path — those three states are where junior work falls apart).

Identify every layer the feature touches — database, data-access, business logic, API, frontend, infra/config, jobs/webhooks — and note which are risky. Flag anything hitting the Do-Not-Vibe overlay (payment math, webhooks, migrations, auth/reset, production secrets) so it gets human hands, not an unattended queue.

Sequence the work into tasks and order them by dependency. Data layer usually first (everything depends on it); UI polish usually last. Aim for an order where each step leaves the app in a runnable state.

The output of this phase is the architecture/logic map

Phase 2 — Environment and branch setup

Housekeeping that prevents a whole class of "works on my machine" pain.

- **Sync with the mainline.** Pull the latest `main`/`develop`` so you branch from current reality, not last week's.
- **Create a feature branch** with a clear name (`feat/revenue-dashboard``). Never build features directly on the shared branch.
- **Get the app running locally and green** *before* you change anything — run the existing tests, apply any pending migrations, install new dependencies. If it's broken before you start, you won't know whether you broke it.
- **Set up a feature flag** if the feature is large, risky, or will ship in pieces. A flag lets you merge incomplete work safely (it's off in production) and roll out gradually later.
- **Seed data / fixtures** you'll need to see the feature work locally.

Phase 3 — Scaffold the vertical slice

Do not build one layer fully and then the next. Instead build a *thin vertical slice* that pierces every layer and works end to end — one field, one row, one button — then thicken it.

Why vertical, not horizontal: a horizontal approach (finish the whole database, then the whole backend, then the whole frontend) means nothing actually runs until the very end, and integration problems all hit you at once, late.

A vertical slice gives you a working (if trivial) feature early, proves the layers connect, and surfaces contract mismatches immediately.

Build the slice bottom-up, layer by layer, using generators where they exist:

1. **Data layer.** Create the migration and the model/entity for the minimal schema — even just one table with a couple of columns. Run the migration. (Generators: `make:model -m``, `nest g``, Prisma `migrate dev``, etc.) Migrations are a Do-Not-Vibe item — write and review these deliberately.
2. **Data-access layer.** The repository/query layer that reads and writes that model. One method: fetch the thing.
3. **Business-logic / service layer.** The place your actual rules live, separate from HTTP and DB concerns. For the slice, it can just pass data through. Keeping logic here (not in controllers or components) is what makes it testable and reusable.
4. **API layer.** One route/endpoint: controller/handler, request validation, response serialization (DTO), error format. Wire it to call the service. Now you can hit the endpoint and get real data back.

5. **Frontend data layer.** The API client call and the state/store that holds the response — including loading, empty, and error states from the start.
6. **Frontend UI.** One component rendering that one piece of data, wired into routing/navigation so you can actually reach it.

At the end of Phase 3 you have a feature that does *one true thing* end-to-end: click → request → service → query → database → back up the stack → rendered on screen. Everything from here is thickening, not construction.

Use stubs to keep it running while incomplete: hardcode a value, return a fake list, `TODO` a branch. A stub that returns the right *shape* lets the layer above it be built before the layer below is finished.

This vertical slice is exactly what your Engineer role should produce *one feature at a time*: a complete slice, not a horizontal pile of half-layers.

Phase 4 — Flesh out the slice (the implementation loop)

Now widen the thin path into the full feature. Work in tight loops, one acceptance criterion at a time. The disciplined loop is:

Write a failing test → make it pass → refactor → commit. (Test-first isn't mandatory everywhere, but writing the test close to the code, not "later," is.)

As you fill in, you're adding the things the slice skipped:

- **Every acceptance criterion** from Phase 0, turned into real behavior.
- **Input validation** at the boundary — never trust the client. Validate types, ranges, required fields, and reject bad input with clear errors.
- **Edge cases and the unhappy path** — empty results, nulls, duplicates, concurrent writes, oversized input, the network failing mid-request. This is most of the real work; the happy path is the easy 20%.
- **Error handling** — catch, translate to meaningful errors, never leak stack traces or swallow failures silently.
- **Authorization and permissions** — who is allowed to do this? Check it server-side, on every protected path, not just by hiding the button.
- **Logging and observability** — log the meaningful events and failures so you can debug in production. Add metrics/traces if the stack supports it.
- **Performance considerations** — the query that's fine on 10 rows and dies on 100k. Add the indexes you noted in Phase 1; watch for N+1 queries; paginate lists.

Commit in small, coherent chunks with clear messages. Small commits make review and rollback sane.

Match your verification effort to the change size, per your own rule: contained changes get targeted tests and analyze-only checks; anything touching shared code, migrations, payments, or persisted state earns the full suite and a clean build.

Phase 5 — Testing (the layers)

Testing isn't one thing; it's a pyramid — many cheap fast tests at the bottom, few slow expensive ones at the top.

- **Unit tests** — individual functions/services in isolation, especially the business-logic layer. Fast, numerous, cover the edge cases and branches. Use factories/fixtures to build test data.
- **Integration tests** — layers working together: the API endpoint hitting a real (test) database, the service plus repository. Catches contract and wiring problems units miss.
- **End-to-end tests** — the whole thing through the real interface (Playwright/Cypress on web, integration/widget tests on Flutter), for the critical user journeys. Few of these; they're slow and brittle, so reserve them for the flows that must never break.
- **Manual QA** — actually use the feature. Click the empty state, the error state, the huge-input state. Try to break it. Test on the real target devices/browsers. Automated tests check what you thought of; manual testing catches what you didn't.

Phase 6 — Quality gates

Before anyone else looks at it, clean it up:

- **Lint and format** — run the linter and formatter (ESLint/Prettier, `dart format`, etc.). Zero warnings.
- **Type-check** — full type pass green (TypeScript, `dart analyze`, mypy).
- **Self-review** — read your own diff top to bottom as if it were someone else's. Remove dead code, commented-out experiments, stray debug logs, and `TODO`s you actually meant to do. Check naming, remove duplication.
- **Update docs and comments** — the README, API docs, inline comments explaining *why* (not what) for anything non-obvious.

Phase 7 — Review and integration

- **Open a pull request** with a description that lets a reviewer understand it without spelunking: what it does, why, how to test it, screenshots/screen recordings for UI, and any risks or follow-ups. Link the ticket.
- **Self-review the PR diff** in the review UI — you'll spot things there you missed in your editor.
- **CI must be green** — automated tests, lint, type-check, build all pass before human review. Don't ask for review on red CI.
- **Address review comments** — respond to each, push fixes, don't take it personally; review is about the code, not you.
- **Merge** using your team's strategy (squash-merge is common — one clean commit per feature). Delete the branch after.

Phase 8 — Pre-release

- **Migration plan** — if there's a schema change, make sure it's backward-compatible or sequenced so the deploy doesn't break the running old version (expand-then-contract: add the new column, deploy code that writes both, backfill, then remove the old — never drop-and-recreate on a live DB).
- **Feature-flag rollout plan** — off by default, then on for you, then a small %, then everyone.
- **Deploy to staging** and smoke-test the real feature in a production-like environment. Bugs that never appear locally appear here.

- **Changelog / release notes** updated.

Phase 9 — Release

- **Deploy to production** during a window you can watch, not right before you log off.
- **Roll out gradually** where possible — flag on for a slice of users, or a canary deploy — rather than 100% at once.
- **Watch the dashboards** actively for the first stretch: error rates, latency, logs, the metric this feature was supposed to move. Have the rollback path ready and know exactly how to trigger it.

Phase 10 — Post-release

- **Verify in production** — actually use the live feature as a real user.
- **Monitor** for a day or two — delayed failures, edge cases real users hit that you didn't.
- **Close the ticket**, note anything deferred.
- **Remove the feature flag** once it's stable and fully rolled out, so it doesn't become permanent dead complexity.
- **Log the tech debt** you knowingly took on ("hardcoded the tax rate, revisit"), so it's tracked instead of forgotten.
- **Iterate** based on the metrics and feedback.

The whole thing as a checklist

Frame: problem statement · user story + acceptance criteria · non-goals · spike unknowns · success metrics

Design: data model · API contract · UI sketch (incl. loading/empty/error) · layers touched + risk flags · task order

Setup: sync main · feature branch · runnable + green locally · feature flag · seed data

Scaffold (vertical slice): migration+model → repository → service → endpoint+validation → frontend client+state → one component, wired end-to-end

Flesh out: test→pass→refactor loop · every criterion · validation · edge/error cases · authz · logging · performance · small commits

Test: unit · integration · e2e on critical paths · manual QA

Quality: lint · format · type-check · self-review · docs

Review: PR with context + screenshots · green CI · address comments · merge · delete branch

Pre-release: safe migration plan · flag rollout plan · staging smoke test · changelog

Release: deploy watched · gradual rollout · monitor · rollback ready

Post-release: verify live · monitor · close ticket · remove flag · log debt · iterate

The one idea to keep

If you internalize only one thing: **build thin and vertical, then thicken.** Get one true end-to-end path working through every layer before you make any single layer complete. The generators, the tests, the flags — all of it hangs off that skeleton. A feature that runs and does one real thing on day one is a feature you can steer; a pile of finished-but-disconnected layers is a feature you're only hoping will connect.